

Genome Matcher: Bioinformatics DNA Analysis Tool

Functional Programming Technical Report | 8th Semester

1 Problem Statement and Industrial Motivation

1.1 Context and Motivation

Analyzing DNA sequences to identify mutations is critical for disease diagnosis and personalized medicine. This project demonstrates functional programming through a bioinformatics use case addressing challenges in medical software: high reliability requirements, parallel processing needs, complex type-safe data structures, deterministic outputs, and real-time processing demands.

1.2 Genome Matcher Implementation

- 1) Mutation detection via recursive comparison of reference and patient DNA sequences
- 2) Risk classification (Benign/Low/Medium/High/Critical) based on position and mutation type
- 3) Protein translation using genetic code lookup (64 codon patterns)
- 4) Parallel processing with `parMap` for multi-core utilization (3x speedup demonstrated)
- 5) Scotty-based web interface with file upload and JSON responses
- 6) Pure functional core with immutable data structures

2 Functional Design with Type Signatures & Main Functions

2.1 Core Algebraic Data Types (ADTs)

```
data Base = A | C | G | T deriving (Show, Eq, Generic, NFData)
```

```
type DNA = [Base]
```

```
data AminoAcid = Phe | Leu | Ser | Tyr | ... | STOP
```

```
type Protein = [AminoAcid]
```

```
data Mutation = Mutation { position :: Int, original :: Base, current :: Base, severity :: RiskLevel }
```

```
data RiskLevel = Benign | Low | Medium | High | Critical
```

```
type AnalysisReport = (String, [Mutation], Double, Protein)
```

2.2 Main Function Signatures

```
analyzePatient :: DNA -> (String, DNA) -> AnalysisReport
```

```
analyzeMultiplePatients :: DNA -> [(String, DNA)] -> [AnalysisReport]
```

```
findMutations :: DNA -> DNA -> [Mutation]
```

```
assessRisk :: Int -> Base -> Base -> RiskLevel
```

```
calculateRiskScore :: [Mutation] -> Double
```

```
dnaToProtein :: DNA -> Protein
```

```
codonLookup :: (Base, Base, Base) -> AminoAcid
```

```
parseDNAString :: String -> DNA
```

2.3 Data Pipeline



3 Functional Programming Concepts Implementation

3.1 Pure Functions

Functions with no side effects. Output depends only on inputs.

```
findMutations :: DNA -> DNA -> [Mutation]
findMutations ref pat = go 0 ref pat
  where
    go i (r:rs) (p:ps) | r == p = go (i+1) rs ps
                      | otherwise = Mutation i r p (assessRisk i r p) : go (i+1) rs ps
    go _ _ _ = []
```

Benefits - Referential transparency, easy testing, thread-safe, reproducible.

3.2 Recursion

Iterates via self-calling functions without imperative loops.

```
dnaToProtein :: DNA -> Protein
dnaToProtein (b1:b2:b3:rest) = codonLookup (b1, b2, b3) : dnaToProtein rest
dnaToProtein _ = []
```

Benefits - Mathematical correctness, tail-call optimization, composability.

3.3 Algebraic Data Types

Sum and product types for type-safe domain modeling.

```
data RiskLevel = Benign | Low | Medium | High | Critical
assessRisk :: Int -> Base -> Base -> RiskLevel
assessRisk idx ref mut
  | idx < 5 = Critical
  | ref == A && mut == T = High
  | otherwise = Low
```

Benefits - Type safety, exhaustive pattern matching, compile-time verification.

3.4 Higher-Order Functions

Functions accepting/returning functions.

```
parseDNString :: String -> DNA
parseDNString raw = map charToBase $ filter (`elem` "ACGTacgt") raw
```

Benefits - Code reuse, declarative style, composability.

3.5 Fold Operations

Reduce lists using accumulator.

```
calculateRiskScore :: [Mutation] -> Double  
calculateRiskScore = foldl (\acc m -> acc + riskValue (severity m)) 0.0
```

Benefits - Abstraction, parallelizable, clear intent.

3.6 Pattern Matching

Decompose structures and bind variables.

```
codonLookup :: (Base, Base, Base) -> AminoAcid  
codonLookup (A,T,G) = Met  
codonLookup (C,T,_) = Leu -- Wildcard patterns
```

Benefits - Concise, wildcards, compiler warnings.

3.7 Immutability

Data immutable after creation.

Benefits - Thread-safe, no defensive copying.

3.8 Type Safety

Compiler verifies types at compile time. ****Benefits****: No null pointers, invalid states impossible.

3.9 Function Composition

```
analyzePipeline = dnaToProtein . findMutations ref . parseDNAString
```

Benefits - Modular, reusable, clear pipeline.

4 Expected Outputs and Possible Extensions

4.1 Output Format

- Mutation list with positions (Position 2: A→T, High Risk)
- Risk score (0-100+) and classification
- Protein sequence (Met-Leu-Ser-Phe-STOP)
- Clinical recommendations and processing metrics

4.2 Possible Extensions

- 1) Insertion/deletion (indel) detection, multiple reference genome comparison
- 2) Machine learning-based risk prediction
- 3) PDF/CSV export generation, real-time genome streaming with lazy evaluation
- 4) Persistent database storage (PostgreSQL)
- 5) Distributed computing across clusters
- 6) GPU-accelerated sequence alignment

- 7) Interactive web visualization dashboard
- 8) Integration with public genomic databases (NCBI, Ensembl).

Output Image:

5 Why FP Improves Reliability and Concurrency

5.1 Reliability Benefits:

- 1) Referential Transparency: Same inputs always produce same outputs, ensuring reproducible medical results
- 2) No Mutable State: Eliminates race conditions, temporal coupling, and aliasing bugs
- 3) Type Safety: Compile-time error detection; Maybe types enforce null-safety
- 4) Compile-Time Verification: Pattern match exhaustivity and type correctness

5.2 Concurrency Implementation:

Parallel processing demonstrates FP advantages. Converting sequential to parallel requires only replacing map with parMap—no locks or synchronization:

```
-- Sequential: map (analyzePatient ref) patients
-- Parallel: parMap rdeepseq (analyzePatient ref) patients
analyzeMultiplePatients :: DNA -> [(String, DNA)] -> [AnalysisReport]
analyzeMultiplePatients refDNA patients =
    parMap rdeepseq (analyzePatient refDNA) patients
```

Performance: 3x speedup (0.15s → 0.05s) on 3 patients using 8 cores.

5.3 Key Advantages:

- 1) It provides automatic thread safety since pure functions inherently thread-safe.
- 2) There are no race conditions therefore immutable data cannot be corrupted.
- 3) There are no Deadlocks, therefore no locks are required.
- 4) Linear scalability can be applied which means more cores are proportional to speedup.
- 5) Composable Concurrency: Nested parallelism works seamlessly.

5.4 Medical Software Impact:

- 1) Patient safety can be certified therefore deterministic analysis prevents misdiagnosis.
- 2) Scalability supports cloud deployment and distributed systems.
- 3) Maintainability can be ensured since pure functions independently testable; type system prevents breaking changes

5.5 Comparison with Imperative:

Imperative requires manual threads, locks, and mutexes with complex error handling (try-catch, null checks). Functional provides automatic parallelization, type-safe errors (Maybe/Either), and simple testing. Result: FP eliminates bug classes while enabling effortless concurrency.